

Axios concept explained



You said:

axios all concept explain

ChatGPT said:

1 What is Axios?

Axios is a **JavaScript library** used to make **HTTP requests** from the browser or Node.js.

It helps you **communicate with APIs** easily.

👉 Example:

- Get data from a server
- Send data to a server
- Update data
- Delete data

Axios works with **Promises** and supports **async/await**.

✳️ Axios is commonly used in frameworks like

- React
- Vue.js

- Angular

2 Installing Axios

Using npm

```
Bash
npm install axios
```

Using CDN

```
HTML
<script
src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js">
</script>
```

3 Basic Syntax

```
JavaScript
axios({
  method: "get",
  url: "https://api.example.com/users"
})
.then(response => {
  console.log(response.data);
})
.catch(error => {
  console.log(error);
});
```

4 Axios HTTP Methods

Axios supports all **HTTP verbs**.

Method	Use
--------	-----

GET	Fetch data
-----	------------

POST	Send data
------	-----------

PUT	Update data
-----	-------------

PATCH	Update partial data
-------	---------------------

Method	Use
--------	-----

DELETE	Delete data
--------	-------------

5 GET Request (Fetch Data)

```
JavaScript
axios.get("https://api.example.com/users")
  .then(res => {
    console.log(res.data);
  })
  .catch(err => {
    console.log(err);
  });
```

Using Async / Await

```
JavaScript
async function getUsers() {
  try {
    let res = await
    axios.get("https://api.example.com/users");
    console.log(res.data);
  } catch(err) {
    console.log(err);
  }
}

getUsers();
```

6 POST Request (Send Data)

```
JavaScript
axios.post("https://api.example.com/users", {
  name: "Aamir",
  age: 22
})
  .then(res => {
    console.log(res.data);
  });
```

7 PUT Request (Update Data)

```
JavaScript
axios.put("https://api.example.com/users/1", {
  name: "Aamir Updated"
});
```

8 DELETE Request

```
JavaScript
axios.delete("https://api.example.com/users/1");
```

9 Axios Response Object

When Axios gets a response, it returns an object.

Example:

```
JavaScript
axios.get("https://api.example.com/users")
  .then(res => {
    console.log(res);
  });
```

Response contains:

<code>res.data</code>	→ actual data
<code>res.status</code>	→ status code
<code>res.headers</code>	→ headers
<code>res.config</code>	→ request configuration

Example:

```
JavaScript
console.log(res.data);
```

10 Axios Error Handling

```
JavaScript
axios.get("https://api.example.com/users")
  .then(res => {
    console.log(res.data);
  })
  .catch(error => {
    console.log(error.message);
  });
```

Better error handling:

```
JavaScript
catch(error => {
  if (error.response) {
    console.log(error.response.status);
  } else {
    console.log(error.message);
  }
});
```

1 1 Sending Query Parameters

```
JavaScript
axios.get("https://api.example.com/users", {
  params: {
    id: 1,
    name: "Aamir"
  }
});
```

URL becomes:

```
https://api.example.com/users?id=1&name=Aamir
```

1 2 Axios Headers

Headers send extra information to the server.

```
JavaScript
axios.get("https://api.example.com/users", {
  headers: {
    Authorization: "Bearer token123"
  }
});
```

1 3 Axios Interceptors (Important)

Interceptors allow you to **modify request or response before they are handled**.

Request Interceptor

```
JavaScript
axios.interceptors.request.use(config => {
  console.log("Request sent");
  return config;
});
```

Response Interceptor

```
JavaScript
axios.interceptors.response.use(response => {
  console.log("Response received");
  return response;
});
```

1 4 Axios Instance

Create reusable axios configuration.

```
JavaScript
const api = axios.create({
  baseURL: "https://api.example.com",
  timeout: 5000
});

api.get("/users");
```

1 5 Cancel Request

```
JavaScript
const controller = new AbortController();

axios.get("/users", {
  signal: controller.signal
});

controller.abort();
```

1 6 Axios vs Fetch

Feature	Axios	Fetch
JSON auto parsing	✓ Yes	✗ No
Error handling	Better	Manual
Interceptors	✓ Yes	✗ No
Timeout	Built-in	Manual

1 7 Real Example API

```
JavaScript
async function getJokes() {
  let res = await axios.get(
    "https://official-joke-api.appspot.com/random_joke"
  );

  console.log(res.data);
}

getJokes();
```

Output:

```
{
  setup: "...",
  punchline: "..."
}
```

1 8 Axios in React Example

```
JavaScript
import axios from "axios";
import { useEffect } from "react";

function App() {

  useEffect(() => {
    axios.get("https://api.example.com/users")
      .then(res => console.log(res.data));
  }, []);

}
```

★ Axios Interview Questions

1 What is Axios?

Axios is a **Promise-based HTTP client used to make API requests in JavaScript**.

2 Difference between Axios and Fetch?

Axios automatically converts JSON and provides better error handling.

3 What are Axios interceptors?

Functions that run **before request is sent or after response is received**.

4 What is Axios instance?

A **custom Axios object with predefined configuration**.

✓ Short Definition

Axios = **Library to communicate with APIs easily using HTTP requests**.

💡 If you want, I can also explain:

- **Axios vs Fetch (deep comparison)**
- **Axios real projects (React + API)**
- **Axios interview coding questions**

- **10 Axios practice programs**

Just tell me and I'll show you 🚀

You said:

using axios

ChatGPT said:

Using Axios

Axios is used to **send HTTP requests to APIs** and get data from servers. You can use it in **JavaScript, Node.js, or frontend frameworks like React**.

1 Step 1: Install Axios

Using npm

```
Bash
npm install axios
```

Import Axios

```
JavaScript
import axios from "axios";
```

Or with CDN in HTML:

```
HTML
<script
src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js">
</script>
```

2 Basic Axios Request

```
JavaScript
axios({
  method: "get",
  url: "https://api.example.com/users"
})
.then(function(response) {
  console.log(response.data);
})
.catch(function(error) {
  console.log(error);
});
```

3 Using Axios GET Request

Used to **fetch data from API**.

```
JavaScript
axios.get("https://jsonplaceholder.typicode.com/posts")
.then((response) => {
  console.log(response.data);
})
.catch((error) => {
  console.log(error);
});
```

Output → API data

4 Using Axios POST Request

Used to **send data to server**.

```
JavaScript
axios.post("https://jsonplaceholder.typicode.com/posts", {
  title: "Hello",
  body: "This is a post",
  userId: 1
})
.then((response) => {
  console.log(response.data);
});
```

5 Using Axios with **async / await**

Modern JavaScript uses **async-await**.

```
JavaScript
async function getData() {
  try {
    const response = await axios.get(
      "https://jsonplaceholder.typicode.com/users"
    );

    console.log(response.data);

  } catch (error) {
    console.log(error);
  }
}

getData();
```

6 Using Axios with Parameters

```
JavaScript
axios.get("https://jsonplaceholder.typicode.com/posts", {
  params: {
    userId: 1
  }
})
.then((res) => {
  console.log(res.data);
});
```

URL becomes:

```
/posts?userId=1
```

7 Using Axios Headers

Headers send **extra information**.

```
JavaScript
axios.get("https://api.example.com/users", {
  headers: {
    Authorization: "Bearer token123"
  }
});
```

8 Axios Response Object

When request is successful:

```
JavaScript
axios.get("https://api.example.com/users")
  .then(res => {
    console.log(res);
  });
```

Important properties:

res.data	→ actual API data
res.status	→ status code
res.headers	→ response headers
res.config	→ request configuration

9 Handling Errors

```
JavaScript
axios.get("https://wrong-api.com")
  .catch(error => {

    if(error.response) {
      console.log(error.response.status);
    }

    else{
      console.log(error.message);
    }

  });
```

10 Example: Real API Request

```
JavaScript
async function getJoke() {

  const res = await axios.get(
    "https://official-joke-api.appspot.com/random_joke"
  );

  console.log(res.data.setup);
  console.log(res.data.punchline);

}

getJoke();
```

Example Output:

```
Why did the computer show up at work late?
It had a hard drive!
```

1 1 Axios in React Example

```
JavaScript
import axios from "axios";
import { useEffect } from "react";

function App() {

  useEffect(() => {

    axios.get("https://jsonplaceholder.typicode.com/users")
      .then(res => {
        console.log(res.data);
      });

  }, []);

}
```

Simple Flow of Axios

```
Client (Browser)
  ↓
Axios Request
  ↓
API Server
  ↓
Response
  ↓
Axios returns Data
```

✅ In one line:

Axios is used to **send HTTP requests and receive API data easily in JavaScript.**

If you want, I can also explain **advanced Axios topics** that are very important for interviews and real projects:

- Axios **Interceptors**
- Axios **Instance**
- Axios **BaseURL**
- Axios **Timeout**
- Axios **Cancel Request**
- **Axios vs Fetch**

Just tell me and I'll explain them **step-by-step with examples.** 🚀

You said:

dog picture api

ChatGPT said:

A popular **dog image API** you can use in JavaScript is the **Dog CEO Dog API**. It provides **random dog pictures** and **dog breed images**.

API base URL:

```
https://dog.ceo/api
```

1 Random Dog Picture API

Endpoint:

```
https://dog.ceo/api/breeds/image/random
```

Example response:

```
JSON
{
  "message": "https://images.dog.ceo/breeds/hound-afghan/n02088094_1003.jpg",
  "status": "success"
}
```

message → image URL

status → request status

2 Using the API with Axios

```
JavaScript
import axios from "axios";

async function getDog() {
  try {
    const res = await
axios.get("https://dog.ceo/api/breeds/image/random");
    console.log(res.data.message);
  } catch (error) {
    console.log(error);
  }
}

getDog();
```

Output:

```
https://images.dog.ceo/breeds/hound-afghan/....
```

3 Display Dog Image on Website

HTML

HTML

```
<button id="btn">Get Dog</button>
<img id="dogImg" width="300">
```

JavaScript

JavaScript

```
const btn = document.querySelector("#btn");
const img = document.querySelector("#dogImg");

btn.addEventListener("click", async () => {

  let res = await
  axios.get("https://dog.ceo/api/breeds/image/random");

  img.src = res.data.message;

});
```

👉 When you click the button → a **new random dog image appears**.

4 Get Multiple Dog Images

Endpoint:

```
https://dog.ceo/api/breeds/image/random/5
```

Axios example:

```
JavaScript
axios.get("https://dog.ceo/api/breeds/image/random/5")
.then(res => {
  console.log(res.data.message);
});
```

This returns **5 dog image URLs**.

5 Get Images by Dog Breed

Example:

```
https://dog.ceo/api/breed/hound/images
```


Axios:

```
JavaScript
axios.get("https://dog.ceo/api/breed/hound/images")
  .then(res => {
    console.log(res.data.message);
  });
```

✓ Summary

Dog picture API (Dog CEO API) provides:

- Random dog images
- Multiple dog images
- Images by breed
- Completely free API

💡 If you want, I can also show a **small JavaScript project** (very common interview practice):

"Random Dog Generator App"

(button → new dog picture appears). 🐕

You said:

```
const btn = document.querySelector("#btn"); const img =
document.querySelector("#dogImg"); btn.addEventListener("click", async () => {
let res = await axios.get("https://dog.ceo/api/breeds/image/random"); img.src
= res.data.message; });with html
```

ChatGPT said:

Here is the **complete working example (HTML + JavaScript)** using **Axios** and the **Dog CEO Dog API** to show a random dog image when a button is clicked.

Complete HTML + JS Example

```
HTML
<!DOCTYPE html>
<html>
<head>
  <title>Random Dog Generator</title>
</head>

<body>

  <h2>Random Dog Image 🐕</h2>

  <button id="btn">Get Dog Image</button>
  <br><br>

  <img id="dogImg" width="300">

  <!-- Axios CDN -->
  <script
src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js">
</script>

  <script>

    const btn = document.querySelector("#btn");
    const img = document.querySelector("#dogImg");

    btn.addEventListener("click", async () => {

      let res = await
      axios.get("https://dog.ceo/api/breeds/image/random");

      img.src = res.data.message;

    });

  </script>

</body>
</html>
```

How This Works

1 Button

```
HTML
<button id="btn">Get Dog Image</button>
```

User clicks the button.

2 Image Element

```
HTML
<img id="dogImg" width="300">
```

This is where the **dog picture will appear**.

3 Axios Request

```
JavaScript
let res = await
axios.get("https://dog.ceo/api/breeds/image/random");
```

This sends a **GET request to the dog API**.

4 Display Image

```
JavaScript
img.src = res.data.message;
```

API returns an image URL → it is placed inside **img.src**.

Result

When you click the button:

```
Click Button
  ↓
Axios API Request
  ↓
Dog API returns image URL
  ↓
Image appears on screen
```

✓ This is a **very common beginner API project** and also used when learning APIs in **JavaScript**.

If you want, I can also show you a **better version of this project**:

- Loading spinner while image loads

- Error handling (try/catch)
- New dog image every click
- Better UI with CSS

This becomes a **mini API project** developers often build while learning APIs.



You said:

sending headers with api

ChatGPT said:

Sending Headers with an API Request

When making API requests (for example with **Axios**), **headers** are used to send **extra information to the server**.

Headers can include things like:

- Authorization tokens
- Content type (JSON, text, etc.)
- API keys
- Language preferences

They are part of the **HTTP request protocol** used by the web and APIs.

1 Basic Axios Request with Headers

```
JavaScript
axios.get("https://api.example.com/users", {
  headers: {
    Authorization: "Bearer token123"
  }
})
.then(res => {
  console.log(res.data);
});
```

Explanation

headers → extra information sent to the server
Authorization → used for authentication
Bearer token123 → access token

2 Multiple Headers Example

```
JavaScript
axios.get("https://api.example.com/data", {
  headers: {
    Authorization: "Bearer myToken123",
    "Content-Type": "application/json",
    "Custom-Header": "HelloAPI"
  }
})
.then(res => {
  console.log(res.data);
});
```

Common headers:

Header	Purpose
Authorization	Login / security token
Content-Type	Format of data
Accept	Type of response expected
User-Agent	Client information

3 Headers with POST Request

```
JavaScript
axios.post(
  "https://api.example.com/users",
  {
    name: "Aamir",
    age: 22
  },
  {
    headers: {
      "Content-Type": "application/json"
    }
  }
)
.then(res => {
  console.log(res.data);
});
```

Structure:

```
axios.post(URL, DATA, CONFIG)
```

4 Headers with Async / Await

```
JavaScript
async function getData() {
  try {
    const res = await
    axios.get("https://api.example.com/users", {
      headers: {
        Authorization: "Bearer token123"
      }
    });

    console.log(res.data);

  } catch (err) {
    console.log(err);
  }
}

getData();
```

5 Setting Default Headers

You can set headers globally for **all requests**.

```
JavaScript
axios.defaults.headers.common["Authorization"] = "Bearer token123";
```

Now every request will include this header.

6 Creating Axios Instance with Headers

```
JavaScript
const api = axios.create({
  baseURL: "https://api.example.com",
  headers: {
    Authorization: "Bearer token123"
  }
});

api.get("/users");
```

This is very common in projects built with frameworks like **React**.

7 Example Using a Public API Key

Some APIs require keys in headers.

Example format:

```
JavaScript
axios.get("https://api.example.com/weather", {
  headers: {
    "x-api-key": "123456789"
  }
});
```

 Simple Flow

```
Client (Browser)
  ↓
Axios Request
  ↓
Headers attached
  ↓
API Server
  ↓
Response
```

 Short Definition

Headers = extra information sent with an API request.

Examples:

```
Authorization
Content-Type
API Keys
Cookies
```

💡 If you want, I can also explain **very important interview concepts** related to APIs:

- HTTP headers **in detail**
- **Authorization vs Authentication**
- **CORS headers**
- **Request vs Response headers**
- Real **API debugging in Postman**

These are **very important for MERN developers**.

You said:

```
const url = "https://icanhazdadjoke.com/"; async function getJokes() { try {
const config = { headers: { Accept: "application/json" } }; let res = await
axios.get(url, config); console.log(res.data); } catch (err) { console.log(err); }
```


ChatGPT said:

Your code is trying to get a joke from the **icanhazdadjoke API** using **Axios** and **headers**.

Your code has some **syntax mistakes**, so here is the **correct version**.

Correct Code

```
JavaScript
const url = "https://icanhazdadjoke.com/";

async function getJokes() {

  try {

    const config = {
      headers: {
        Accept: "application/json"
      }
    };

    let res = await axios.get(url, config);

    console.log(res.data);

  } catch (err) {

    console.log(err);

  }

}

getJokes();
```

Important Part (Headers)

```
JavaScript
const config = {
  headers: {
    Accept: "application/json"
  }
};
```

This header tells the API:

```
Send response in JSON format
```

Because the **icanhazdadjoke API** can return data in **different formats**:

Header

Response Format

Accept: text/html

HTML page

Accept: application/json JSON data



API Response Example

JSON

```
{
  "id": "R7UfaahVfFd",
  "joke": "Why did the computer show up at work late? It had a
hard drive.",
  "status": 200
}
```

To print only the joke:

```
JavaScript
console.log(res.data.joke);
```



Example Output

```
Why did the computer show up at work late?
It had a hard drive.
```

Full Working HTML Example

```
HTML
<!DOCTYPE html>
<html>
<head>
<title>Dad Joke Generator</title>
</head>

<body>

<button onclick="getJokes()">Get Joke</button>

<script
src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js">
</script>

<script>

const url = "https://icanhazdadjoke.com/";

async function getJokes(){

  try{

    const config = {
      headers: { Accept: "application/json" }
    };

    let res = await axios.get(url, config);

    alert(res.data.joke);

  }catch(err){
    console.log(err);
  }

}

</script>

</body>
</html>
```

Very Important Concept

```
JavaScript
axios.get(url, config)
```

Structure:

```
axios.get(URL, CONFIG)
```

CONFIG contains:

```
headers  
params  
timeout  
auth
```

✅ Simple Meaning

Headers are used to **tell the server how we want the response.**

Example:

```
Accept: application/json
```

means

```
Server → Send data in JSON format
```

If you want, I can also show you **3 very famous API practice projects** developers build when learning APIs:

- 1 Random Joke Generator 🤡
- 2 Random Dog Image Generator 🐕
- 3 Random Cat Facts Generator 🐈

These help you master **Axios + APIs + Async/Await** very fast. 🚀

You said:

```
document.addEventListener('DOMContentLoaded', () => { const searchBtn =  
document.getElementById('searchbtn'); const countryInput =  
document.getElementById('countryInput'); const resultsDiv =  
document.getElementById('results'); searchBtn.addEventListener('click', async ()  
=> { const country = countryInput.value.trim(); if (!country) { showError('Please  
enter a country name!'); return; } try { const universities = await  
getUniversities(country); displayUniversities(universities); } catch (error) {  
console.error('Error:', error); showError('Failed to fetch universities. Please try
```

```
again.}); }); async function getUniversities(country) { const url =
http://universities.hipolabs.com/search?
country=${encodeURIComponent(country)}; const response = await
axios.get(url); return response.data; } function displayUniversities(universities) {
resultsDiv.innerHTML = ""; if (universities.length === 0) { showError("No
universities found for this country."); return; } universities.forEach(university => {
const universityCard = document.createElement('div');
universityCard.className = 'university-card'; universityCard.innerHTML =
<h3>${university.name}</h3> <p><strong>Country:</strong>
${university.country}</p> ${university.web_pages?.length ? <p>
<strong>Website:</strong> <a href="${university.web_pages[0]}"
target="_blank">${university.web_pages[0]}</a></p> : ""}
${university.domains?.length ? <p><strong>Domain:</strong>
${university.domains[0]}</p> : ""} ; resultsDiv.appendChild(universityCard); });
} function showError(message) { resultsDiv.innerHTML = <p class="error-
message">${message}</p>; } }); for html
```

ChatGPT said:

Below is the **HTML file** that works with your JavaScript code and uses **Axios** to fetch data from the **Hipolabs Universities API**.

This page lets a user **enter a country name and see universities from that country**.

Complete HTML

```
HTML
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>University Finder</title>

<style>

body{
    font-family: Arial;
    background:#f4f4f4;
    text-align:center;
}

.container{
    width:80%;
    margin:auto;
}

input{
    padding:10px;
    width:250px;
}

button{
    padding:10px 15px;
    cursor:pointer;
}

#results{
    margin-top:20px;
}

.university-card{
    background:white;
    border:1px solid #ddd;
    padding:15px;
    margin:10px;
    border-radius:5px;
}

.error-message{
    color:red;
}

</style>
</head>
```

```
<body>

<div class="container">

<h1>University Search</h1>

<input
type="text"
id="countryInput"
placeholder="Enter Country Name"
/>

<button id="searchbtn">Search</button>

<div id="results"></div>

</div>

<!-- Axios CDN -->
<script
src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js">
</script>

<script>

document.addEventListener('DOMContentLoaded', () => {

    const searchBtn = document.getElementById('searchbtn');
    const countryInput =
document.getElementById('countryInput');
    const resultsDiv = document.getElementById('results');

    searchBtn.addEventListener('click', async () => {

        const country = countryInput.value.trim();

        if(!country){
            showError('Please enter a country name!');
            return;
        }

        try{

            const universities = await
getUniversities(country);
            displayUniversities(universities);

        }catch(error){

            console.error(error);
            showError('Failed to fetch universities.');
```

```
async function getUniversities(country) {

    const url = `http://universities.hipolabs.com/search?
country=${encodeURIComponent(country)}`;

    const response = await axios.get(url);

    return response.data;

}

function displayUniversities(universities) {

    resultsDiv.innerHTML = "";

    if(universities.length === 0) {
        showError('No universities found');
        return;
    }

    universities.forEach(university => {

        const universityCard =
document.createElement('div');
        universityCard.className = "university-card";

        universityCard.innerHTML = `
<h3>${university.name}</h3>
<p><strong>Country:</strong> ${university.country}
</p>
<p><strong>Website:</strong>
<a href="${university.web_pages[0]}"
target="_blank">
    ${university.web_pages[0]}
</a></p>
`;

        resultsDiv.appendChild(universityCard);

    });

}

function showError(message) {
    resultsDiv.innerHTML = `<p class="error-
message">${message}</p>`;
}

});

</script>
```



```
</body>  
</html>
```

How the Project Works

1 User enters a country

```
India  
USA  
Canada  
Germany
```

2 Axios sends API request

```
http://universities.hipolabs.com/search?country=India
```

3 API returns data

Example:

```
JSON  
[  
  {  
    "name": "University of Delhi",  
    "country": "India",  
    "web_pages": ["http://www.du.ac.in"]  
  }  
]
```

4 JavaScript displays universities on the page

Each university appears like:

```
University of Delhi  
Country: India  
Website: du.ac.in
```

✓ This is a **very good beginner API project** because it teaches:

- API requests
- Async / Await

- Axios
- DOM manipulation
- Error handling

💡 If you want, I can also show you how to **improve this project to a professional level:**

- Add **loading spinner while API loads**
- Add **search on Enter key**
- Add **university count**
- Add **modern card UI**
- Make it look like a **real website project**

That will make this a **great portfolio project for MERN developers.** 🚀

You said:

write code as state base college in india

ChatGPT said:

You can modify your project to **search colleges in India by state.**

We will still use the **Hipolabs Universities API** and make the request with **Axios.**

⚠️ Important: This API only supports **country search**, not state directly.
So we will:

1. Fetch all universities from **India**

2. Filter them in JavaScript based on **state name typed by the user**.



Complete Example (State-Based College

Search)

HTML

```
HTML
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>State Based College Finder</title>

<style>

body{
    font-family: Arial;
    text-align:center;
    background:#f4f4f4;
}

input{
    padding:10px;
    width:250px;
}

button{
    padding:10px 15px;
    cursor:pointer;
}

#results{
    margin-top:20px;
}

.card{
    background:white;
    padding:15px;
    margin:10px auto;
    width:60%;
    border:1px solid #ddd;
    border-radius:5px;
}

.error{
    color:red;
}

</style>
</head>

<body>
```

```
<h2>Find Colleges by State (India)</h2>

<input type="text" id="stateInput" placeholder="Enter State
(ex: Punjab)">
<button id="searchBtn">Search</button>

<div id="results"></div>

<script
src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js">
</script>
<script src="script.js"></script>

</body>
</html>
```

JavaScript (script.js)

```
JavaScript
document.addEventListener("DOMContentLoaded", () => {

  const btn = document.getElementById("searchBtn");
  const stateInput = document.getElementById("stateInput");
  const results = document.getElementById("results");

  btn.addEventListener("click", async () => {

    const state = stateInput.value.trim().toLowerCase();

    if(!state){
      showError("Please enter a state name");
      return;
    }

    try{

      const colleges = await getColleges();
      const filtered = colleges.filter(college =>
        college.name.toLowerCase().includes(state)
      );

      displayColleges(filtered);

    }catch(err){
      showError("Failed to fetch data");
    }

  });

  async function getColleges(){

    const url = "http://universities.hipolabs.com/search?country=India";

    const res = await axios.get(url);

    return res.data;

  }

  function displayColleges(colleges){

    results.innerHTML = "";

    if(colleges.length === 0){
      showError("No colleges found for this state");
      return;
    }

  }

}
```

```
colleges.forEach(college => {

    const div = document.createElement("div");
    div.className = "card";

    div.innerHTML = `
    <h3>${college.name}</h3>
    <p><b>Country:</b> ${college.country}</p>
    <p><b>Website:</b>
    <a href="${college.web_pages[0]}" target="_blank">
    ${college.web_pages[0]}
    </a>
    </p>
    `;

    results.appendChild(div);

});

function showError(msg){
    results.innerHTML = `<p class="error">${msg}</p>`;
}

});
```

How It Works

1 User enters a state

```
Punjab
Delhi
Gujarat
Maharashtra
```

2 Axios fetches Indian universities

```
http://universities.hipolabs.com/search?country=India
```

3 JavaScript filters results

```
JavaScript
college.name.includes(state)
```

4 Matching colleges are displayed on the page.

Example Search

Input:

Punjab

Result:

Punjab Engineering College
Punjab University
Guru Nanak Dev University

✅ This project teaches:

- API requests with Axios
- Filtering data
- DOM manipulation
- Search functionality

💡 If you want, I can also show a **much better version of this project** (used in real portfolios):

- **State dropdown for all 28 Indian states**
- **Search suggestions**
- **College count**
- **Modern UI cards**
- **Fast filtering**

That will make this look like a **real frontend project**. 🚀

ChatGPT can make mistakes. Check important info. See [Cookie Preferences](#).